

Guide d'Installation AirBar Backend - Production

Installation Serveur Local ou Distant (Windows / Linux)

Version: 2.0
Date: 2026-08-16
Testé sur: Windows 10/11, Ubuntu 22.04/24.04

Table des matières

- 1. [Vue d'ensemble](#)
 - 2. [Prérequis](#)
 - 3. [Installation des outils](#)
 - 4. [Configuration initiale](#)
 - 5. [Déploiement](#)
 - 6. [Vérification et tests](#)
 - 7. [Dépannage](#)
 - 8. [Maintenance](#)
-

Vue d'ensemble

Architecture de déploiement





Deux types d'installation

Type	Scénario	Outils requis
Local	Machine de développement ou serveur physique accessible	Git + Docker + Dart/Flutter (optionnel)
Distant	Serveur VPS/Cloud accessible via SSH	Git + Docker uniquement

⚠ Point critique : Fichiers générés par Serverpod

Depuis la version 3.3.1 de Serverpod, les fichiers générés ([lib/src/generated/](#)) DOIVENT être versionnés dans Git pour un déploiement Docker sans Flutter.

Pourquoi ? La génération de code nécessite Flutter, mais Docker utilise une image Dart légère (~200MB vs ~2GB avec Flutter). Les fichiers générés sont donc créés en développement puis versionnés.

Prérequis

Serveur Local (Windows ou Linux)

- **OS:** Windows 10/11 64-bit OU Linux (Ubuntu 22.04+, Debian 11+)
- **RAM:** 4 Go minimum, 8 Go recommandé
- **Disque:** 20 Go libres
- **Droits:** Administrateur (Windows) ou sudo (Linux)
- **Internet:** Connexion stable

Serveur Distant (Linux uniquement)

- **OS:** Ubuntu 22.04+ ou Debian 11+
 - **Accès:** SSH avec clés publiques (recommandé) ou mot de passe
 - **RAM:** 4 Go minimum, 8 Go recommandé
 - **Disque:** 20 Go libres
 - **Ports:** 8080, 8081, 8082 ouverts (pare-feu)
-



Installation des outils

Option A : Windows (Local)

1. Installer Git

1. Télécharger depuis <https://git-scm.com/download/win>
2. Exécuter l'installateur avec les options par défaut
3. Vérifier l'installation :

```
git --version  
# Attendu: git version 2.x.x
```

2. Installer Docker Desktop

1. Télécharger depuis <https://www.docker.com/products/docker-desktop/>
2. Exécuter l'installateur
3. **Important:** Accepter l'utilisation de WSL2 (Windows Subsystem for Linux)
4. Redémarrer Windows si demandé
5. Lancer Docker Desktop et attendre l'état **"Running"** (icône verte)
6. Vérifier l'installation :

```
docker --version  
# Attendu: Docker version 24.x.x  
  
docker compose version  
# Attendu: Docker Compose version v2.x.x
```

⚠ **Si docker compose n'est pas reconnu :** Fermez et rouvrez PowerShell après le démarrage de Docker Desktop.

3. (Optionnel) Installer Dart et Flutter

Nécessaire uniquement si vous modifiez le code serveur localement.

- Flutter SDK: <https://docs.flutter.dev/get-started/install/windows>
 - Dart est inclus avec Flutter
-

Option B : Linux (Local ou Distant)

1. Se connecter au serveur

Serveur local : Ouvrir un terminal

Serveur distant : Se connecter via SSH

```
ssh utilisateur@adresse-ip-serveur
```

2. Installer Git

```
sudo apt update
sudo apt install -y git
git --version
```

3. Installer Docker Engine + Docker Compose Plugin

```
# Installer les dépendances
sudo apt install -y ca-certificates curl gnupg

# Ajouter la clé GPG officielle de Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor \
-o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Ajouter le dépôt Docker
echo \
  "deb [arch=$(dpkg --print-architecture) signed- \
  by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Installer Docker
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

```
# Vérifier l'installation
docker --version
docker compose version
```

4. Configurer les permissions Docker (optionnel mais recommandé)

```
# Ajouter l'utilisateur au groupe docker pour éviter sudo
sudo usermod -aG docker $USER

# Appliquer les changements (déconnexion/reconnexion)
newgrp docker

# Vérifier
docker ps
# Doit fonctionner sans erreur de permission
```



Configuration initiale

1. Récupérer le code source

Windows (PowerShell)

```
# Créer le dossier de travail
cd C:\
mkdir airbar_production
cd airbar_production

# Cloner le dépôt (branche prod uniquement)
git clone -b prod --single-branch
https://github.com/rorophil/airbar_backend.git
cd .\airbar_backend\airbar_backend_server

# Vérifier la structure
dir
```

Linux (Bash)

```
# Créer le dossier de travail
mkdir -p ~/airbar_production
```

```
cd ~/airbar_production

# Cloner le dépôt (branche prod uniquement)
git clone -b prod --single-branch
https://github.com/rorophil/airbar_backend.git
cd airbar_backend/airbar_backend_server

# Vérifier la structure
ls -la
```

💡 **Optimisation :** L'option `--single-branch` télécharge uniquement la branche `prod`, ce qui réduit le temps de clonage et l'espace disque utilisé (on évite de télécharger toutes les branches de développement inutiles sur le serveur de production).

Fichiers attendus :

- `docker-compose.yaml`
- `Dockerfile`
- `config/production.yaml`
- `config/passwords.yaml`
- `.env.example`

2. Vérifier les fichiers générés (CRITIQUE)

```
# Les fichiers suivants DOIVENT exister
ls lib/src/generated/endpoints.dart
ls lib/src/generated/protocol.dart
ls lib/src/generated/protocol.yaml
```

✗ Si ces fichiers n'existent pas :

```
# Vérifier que vous avez bien la dernière version
git pull origin prod

# Vérifier le .gitignore à la racine du workspace
cat ../.gitignore | grep generated
```

La ligne `**/generated/` NE DOIT PAS être présente dans le `.gitignore` principal (à la racine du workspace, pas celui du serveur).

Si les fichiers générés sont absents et que vous avez Dart/Flutter installés :

```
# En développement local seulement
cd .. # Retour à la racine du workspace
serverpod generate
cd airbar_backend_server
```

3. Configurer les secrets de production

Étape 3.1 : Examiner `config/passwords.yaml`

```
# Afficher les mots de passe de production actuels
cat config/passwords.yaml | grep -A 10 "production:"
```

Exemple de sortie :

```
production:
  database: '-g4x7Ke0FV68yDNSi1xDQpirxejC2580'
  serviceSecret: 'k7N45MLCfD2PNbI_IvKc2bYwqn9FM05i'
  emailSecretHashPepper: '9j7p2HCKJevye1Eo8GndwhKSaw7X2SKd'
  jwtHmacSha512PrivateKey: 'QjjCRgRcEwJcjh2sJbxF9n24TLvIpYL'
  jwtRefreshTokenHashPepper: 'XlrfjPF3mW_lfHT355US_dpcVLxgStfc'
```

Étape 3.2 : Créer le fichier `.env`

⚠ CRITIQUE : Les valeurs dans `.env` **DOIVENT** correspondre EXACTEMENT à celles dans `config/passwords.yaml` section `production`.

Windows (PowerShell) :

```
# Copier le template
copy .env.example .env

# Éditer avec notepad
notepad .env
```

Linux (Bash) :

```
# Copier le template
cp .env.example .env
```

```
# Éditer avec nano ou vim  
nano .env
```

Contenu du fichier **.env** :

```
# =====  
# AIRBAR BACKEND – CONFIGURATION PRODUCTION  
# =====  
# ⚠ ATTENTION : Les valeurs ci-dessous DOIVENT  
# correspondre à config/passwords.yaml (section production)  
  
# PostgreSQL – Base de données principale  
# Doit correspondre à : passwords.yaml -> production -> database  
POSTGRES_PASSWORD=-g4x7Ke0FV68yDNSi1xDQpirxejC2580  
  
# Redis – Cache et sessions  
REDIS_PASSWORD=VotreMotDePasseRedisSecurise456!  
  
# Serverpod – Clé partagée entre serveurs  
# Doit correspondre à : passwords.yaml -> production -> serviceSecret  
SERVERPOD_PASSWORD_SHARED=k7N45MLCfD2PNbL_IvKc2bYwqn9FM05i  
  
# Serverpod – Secret de service (même valeur que SHARED)  
SERVERPOD_PASSWORD_SERVICE_SECRET=k7N45MLCfD2PNbL_IvKc2bYwqn9FM05i  
  
# JWT – Pepper pour refresh tokens  
# Doit correspondre à : passwords.yaml -> production ->  
jwtRefreshTokenHashPepper  
SERVERPOD_PASSWORD_JWT_REFRESH_PEPPER=XlrfjPF3mW_lfHT355US_dpcVLxgStfc  
  
# JWT – Clé privée HMAC SHA512  
# Doit correspondre à : passwords.yaml -> production ->  
jwtHmacSha512PrivateKey  
SERVERPOD_PASSWORD_JWT_HMAC_SHA512_PRIVATE_KEY=QjjCRgRcEwJcjwH2sjbxF9n24TLvI  
pYL  
  
# Email – Pepper pour hash des secrets email  
# Doit correspondre à : passwords.yaml -> production ->  
emailSecretHashPepper  
SERVERPOD_PASSWORD_EMAIL_SECRET_HASH_PEPPER=9j7p2HCkJevye1Eo8GndwhKSaw7X2SKd
```

Étape 3.3 : Sécuriser le fichier **.env**

Windows :

```
# Vérifier que le fichier n'est pas versionné  
git status .env  
# Doit afficher : "Untracked files" ou rien (si dans .gitignore)
```

Linux :

```
# Restreindre les permissions (lecture seule pour le propriétaire)
chmod 600 .env

# Vérifier
ls -la .env
# Attendu: -rw----- 1 user user ...
```

4. Vérifier la configuration Serverpod

Examinez `config/production.yaml` :

```
cat config/production.yaml
```

Points à vérifier :

```
apiServer:
  port: 8080
  publicHost: localhost      # ⚠ Remplacer par votre domaine en
production
  publicPort: 8080
  publicScheme: http        # ⚠ Passer à https avec reverse proxy

database:
  host: postgres            # ✅ Ne pas changer (nom du service Docker)
  port: 5432
  name: airbar_backend
  user: postgres
  requireSsl: false        # ⚠ true si certificat SSL configuré

redis:
  enabled: true
  host: redis               # ✅ Ne pas changer (nom du service Docker)
  port: 6379
```



Déploiement

1. Construire et démarrer les services

Windows (PowerShell) :

```
cd C:\airbar_production\airbar_backend\airbar_backend_server

# Build et démarrage en arrière-plan
docker compose -f .\docker-compose.yaml up -d --build

# Suivre les logs en temps réel
docker compose -f .\docker-compose.yaml logs -f
```

Linux (Bash) :

```
cd ~/airbar_production/airbar_backend/airbar_backend_server

# Build et démarrage en arrière-plan
docker compose -f ./docker-compose.yaml up -d --build

# Suivre les logs en temps réel
docker compose -f ./docker-compose.yaml logs -f
```

Sortie attendue (après 30-60 secondes) :

```
✓ Network airbar_backend_server_airbar_network Created
✓ Volume airbar_backend_server_postgres_data Created
✓ Volume airbar_backend_server_redis_data Created
✓ Container airbar_postgres_prod Healthy
✓ Container airbar_redis_prod Healthy
✓ Container airbar_backend_prod Started
```

2. Vérifier l'état des services

Commande :

```
# Windows
docker compose -f .\docker-compose.yaml ps

# Linux
docker compose -f ./docker-compose.yaml ps
```

État attendu :

NAME	STATUS	PORTS
airbar_postgres_prod	Up (healthy)	0.0.0.0:5432->5432/tcp
airbar_redis_prod	Up (healthy)	0.0.0.0:6379->6379/tcp
airbar_backend_prod	Up	0.0.0.0:8080-8082->8080-8082/tcp

- ✅ Tous les services doivent être "Up"
- ✅ PostgreSQL et Redis doivent être "healthy"

3. Appliquer les migrations de base de données

⚠️ À exécuter **UNIQUEMENT** au premier déploiement ou après mise à jour du schéma

Windows (PowerShell) :

```
docker compose -f .\docker-compose.yaml run --rm --entrypoint /bin/sh
airbar_server `
  -c "./server --mode=production --server-id=prod-001 --logging=normal --
role=monolith --apply-migrations"
```

Linux (Bash) :

```
docker compose -f ./docker-compose.yaml run --rm --entrypoint /bin/sh
airbar_server \
  -c "./server --mode=production --server-id=prod-001 --logging=normal --
role=monolith --apply-migrations"
```

Sortie attendue :

```
SERVERPOD version: 3.3.1
runMode: production
serverId: prod-001
applyMigrations: true
SERVERPOD initialized

Applying migrations...
✓ Migration 20260303162143579 applied successfully
✓ Migration 20260303165531206 applied successfully
[...]
All migrations applied successfully
```

✗ Si erreur "password authentication failed" :

Voir section [Dépannage - Problème d'authentification PostgreSQL](#)

4. Redémarrer le serveur

```
# Windows
docker compose -f .\docker-compose.yaml restart airbar_server

# Linux
docker compose -f ./docker-compose.yaml restart airbar_server
```

✓ Vérification et tests

1. Vérifier les logs du serveur

```
# Windows
docker compose -f .\docker-compose.yaml logs --tail 100 airbar_server

# Linux
docker compose -f ./docker-compose.yaml logs --tail 100 airbar_server
```

Logs attendus (succès) :

```
SERVERPOD version: 3.3.1
runMode: production
serverId: prod-001
SERVERPOD started
API Server listening on port 8080
Insights Server listening on port 8081
Web Server listening on port 8082
```

✗ **Erreurs à surveiller :**

- password authentication failed → Voir [Dépannage](#)
- No such file or directory: endpoints.dart → Voir [Dépannage](#)
- relation "users" does not exist → Relancer les migrations

2. Tester l'API (Health Check)

Depuis le serveur :

```
curl http://localhost:8080/
```

Depuis un autre poste (si pare-feu ouvert) :

```
curl http://adresse-ip-serveur:8080/
```

Réponse attendue :

```
{"status":"ok","version":"3.3.1"}
```

3. Accéder à Serverpod Insights (monitoring)

Ouvrir dans un navigateur :

```
http://localhost:8081  
# ou  
http://adresse-ip-serveur:8081
```

Page attendue : Dashboard Serverpod Insights avec graphiques de performances

4. Vérifier les données de base

Connexion à PostgreSQL :

```
# Windows  
docker compose -f .\docker-compose.yaml exec postgres psql -U postgres -d  
airbar_backend  
  
# Linux  
docker compose -f ./docker-compose.yaml exec postgres psql -U postgres -d  
airbar_backend
```

Commandes SQL de vérification :

```
-- Lister les tables
\dt

-- Vérifier les utilisateurs
SELECT * FROM serverpod_user_info;

-- Quitter
\q
```

Dépannage

Problème d'authentification PostgreSQL

Symptôme :

```
ERROR: password authentication failed for user "postgres"
```

Cause : Le mot de passe dans `.env` ne correspond pas au mot de passe configuré dans PostgreSQL.

Solution 1 : Réinitialiser PostgreSQL (RECOMMANDÉ au premier déploiement)

```
# 1. Arrêter tous les services
docker compose down

# 2. Supprimer le volume PostgreSQL (⚠ PERTE DE DONNÉES)
docker volume rm airbar_backend_server_postgres_data

# 3. Vérifier le fichier .env
cat .env | grep POSTGRES_PASSWORD
# Doit correspondre à passwords.yaml -> production -> database

# 4. Redémarrer avec le bon mot de passe
docker compose up -d

# 5. Attendre que PostgreSQL soit healthy (15-20 secondes)
docker compose ps

# 6. Relancer les migrations
docker compose run --rm --entrypoint /bin/sh airbar_server \
```

```
-c "./server --mode=production --server-id=prod-001 --logging=normal --  
role=monolith --apply-migrations"
```

Solution 2 : Modifier le mot de passe PostgreSQL existant

```
# 1. Se connecter à PostgreSQL (avec l'ancien mot de passe si possible)  
docker compose exec postgres psql -U postgres  
  
# 2. Changer le mot de passe  
ALTER USER postgres WITH PASSWORD 'nouveau-mot-de-passe-depuis-env';  
\q  
  
# 3. Mettre à jour .env avec le nouveau mot de passe  
nano .env # ou notepad .env sur Windows  
  
# 4. Redémarrer le serveur  
docker compose restart airbar_server
```

Fichiers générés manquants

Symptôme :

```
Error when reading 'lib/src/generated/endpoints.dart': No such file or  
directory
```

Cause : Les fichiers générés par Serverpod ne sont pas présents dans le code source.

Solution :

1. **Vérifier le .gitignore du workspace** (à la racine, pas dans airbar_backend_server)

```
cd ~/airbar_production/airbar_backend # Linux  
cd C:\airbar_production\airbar_backend # Windows  
  
cat .gitignore | grep generated
```

La ligne `**/generated/` NE DOIT PAS être présente. Si elle existe, la supprimer :

```
# Éditer .gitignore et retirer la ligne **/generated/  
nano .gitignore # Linux  
notepad .gitignore # Windows
```

2. Régénérer et versionner les fichiers (en développement local uniquement)

```
# Nécessite Dart + Flutter installés
serverpod generate

# Vérifier que les fichiers sont créés
ls airbar_backend_server/lib/src/generated/

# Ajouter au Git
git add airbar_backend_server/lib/src/generated/
git commit -m "chore: add generated files for production deployment"
git push origin prod
```

3. Sur le serveur distant, récupérer les fichiers

```
git pull origin prod
docker compose up -d --build
```

Le serveur redémarre en boucle

Diagnostic :

```
docker compose logs --tail 200 airbar_server
```

Causes possibles :

1. **Erreur de migration** → Vérifier les logs, corriger le schéma
2. **Problème de connexion BDD** → Vérifier PostgreSQL/Redis
3. **Configuration incorrecte** → Vérifier `production.yaml`
4. **Mémoire insuffisante** → Augmenter la RAM du serveur

Docker Compose non reconnu (Windows)

Symptôme :

```
'docker' is not recognized as an internal or external command
```

Solution :

1. Vérifier que Docker Desktop est **démarré** (icône verte dans la barre des tâches)
2. Fermer et rouvrir PowerShell
3. Redémarrer Windows si nécessaire

Permission denied (Linux)

Symptôme :

```
permission denied while trying to connect to the Docker daemon socket
```

Solution :

```
# Ajouter l'utilisateur au groupe docker
sudo usermod -aG docker $USER

# Appliquer les changements
newgrp docker

# Ou se déconnecter/reconnecter
exit
ssh utilisateur@serveur
```



Maintenance

Arrêter les services

```
# Windows
docker compose -f .\docker-compose.yaml down

# Linux
docker compose -f ./docker-compose.yaml down
```

Redémarrer les services

```
# Windows
docker compose -f .\docker-compose.yaml restart
```

```
# Linux
docker compose -f ./docker-compose.yaml restart
```

Mettre à jour le code

```
# 1. Sauvegarder la base de données
docker exec airbar_postgres_prod pg_dump -U postgres -d airbar_backend -F c
-f /tmp/backup.dump
docker cp airbar_postgres_prod:/tmp/backup.dump ./backup_$(date
+%Y%m%d_%H%M%S).dump

# 2. Récupérer les mises à jour
git pull origin prod

# 3. Rebuild et redémarrage
docker compose down
docker compose up -d --build

# 4. Appliquer les nouvelles migrations (si nécessaire)
docker compose run --rm --entrypoint /bin/sh airbar_server \
  -c "./server --mode=production --server-id=prod-001 --logging=normal --
role=monolith --apply-migrations"
```

Sauvegarder la base de données

```
# Créer le dossier backups
mkdir -p backups

# Dump PostgreSQL
docker exec airbar_postgres_prod pg_dump -U postgres -d airbar_backend -F c
-f /tmp/backup.dump

# Copier localement
docker cp airbar_postgres_prod:/tmp/backup.dump ./backups/airbar_$(date
+%Y%m%d_%H%M%S).dump
```

Restaurer une sauvegarde

```
# Copier le backup dans le conteneur
docker cp ./backups/airbar_20260816_120000.dump
airbar_postgres_prod:/tmp/restore.dump
```

```
# Restaurer
docker exec airbar_postgres_prod pg_restore -U postgres -d airbar_backend -c /tmp/restore.dump
```

Voir les logs en temps réel

```
# Tous les services
docker compose logs -f

# Serveur uniquement
docker compose logs -f airbar_server

# PostgreSQL uniquement
docker compose logs -f postgres
```

Nettoyer les ressources Docker

```
# Supprimer les images non utilisées
docker system prune -a

# Supprimer les volumes orphelins
docker volume prune
```



Sécurité - Checklist Production



AVANT LA MISE EN PRODUCTION RÉELLE

- ☐ **Changer TOUS les mots de passe** dans `config/passwords.yaml` ET `.env`
- ☐ **Utiliser des mots de passe de 32+ caractères** générés aléatoirement
- ☐ **Configurer un pare-feu** (UFW sur Linux, Windows Firewall)
- ☐ **Installer un reverse proxy** (Nginx ou Traefik) avec SSL/TLS
- ☐ **Obtenir un certificat SSL** (Let's Encrypt gratuit)
- ☐ **Configurer les sauvegardes automatiques** (cron job quotidien)
- ☐ **Restreindre l'accès SSH** (clés uniquement, désactiver mot de passe)
- ☐ **Configurer la surveillance** (monitoring, alertes)

- ☐ **Documenter les mots de passe** dans un gestionnaire sécurisé (1Password, Bitwarden)

Générer des mots de passe forts

Linux/macOS :

```
# Générer un mot de passe de 32 caractères  
openssl rand -base64 32
```

Windows (PowerShell) :

```
# Générer un mot de passe de 32 caractères  
-join ((48..57) + (65..90) + (97..122) | Get-Random -Count 32 | ForEach-Object {[char]$_})
```

Configuration pare-feu (Linux)

```
# Installer UFW  
sudo apt install -y ufw  
  
# Configurer les règles  
sudo ufw default deny incoming  
sudo ufw default allow outgoing  
sudo ufw allow ssh  
sudo ufw allow 8080/tcp    # API  
sudo ufw allow 8081/tcp    # Insights  
sudo ufw allow 8082/tcp    # Web  
sudo ufw allow 443/tcp     # HTTPS (avec reverse proxy)  
  
# Activer  
sudo ufw enable  
sudo ufw status
```

Support et ressources

- **Documentation Serverpod** : <https://docs.serverpod.dev>
- **Docker Documentation** : <https://docs.docker.com>

- **Repository GitHub** : https://github.com/rorophil/airbar_backend
 - **Issues** : https://github.com/rorophil/airbar_backend/issues
-



Changelog

Version 2.0 (2026-08-16)

-  Ajout de la procédure complète Windows et Linux
-  Documentation du problème des fichiers générés
-  Correction de l'erreur d'authentification PostgreSQL
-  Ajout de la section sécurité complète
-  Procédures de dépannage étendues

Version 1.0 (2026-08-10)

-  Version initiale
-